

Trading Operations KPI Dashboard

This dashboard is used to track portfolio performance, trader performance, and view market analytics. It was initially developed as a Jupyter notebook, then converted to a Flask application that ran locally on a personal machine, to now being hosted on a server for anyone in the company to view.

When the server is running, the dashboard can be accessed at the configured internal server address on port 5000.

In order to ensure that the dashboard can be maintained beyond the original developer's (Finn's) departure, the setup process has been documented below.

Table of Contents

- [Initial Setup](#)
 - [Cloning the Repository](#)
 - [Setting Up the Virtual Environment](#)
 - [Running the Dashboard Locally](#)
- [Startup Guide](#)
 - [Accessing the Server](#)
 - [Starting the Dashboard](#)
 - [Killing the Dashboard](#)
- [Project Structure & Data Flow](#)
 - [Directory Structure](#)
 - [How Data Flows Through the Dashboard](#)
 - [Stage 1 — Database Connections](#)
 - [Stage 2 — Data Loading](#)
 - [Stage 3 — The results Object](#)
 - [Stage 4 — Chart Rendering](#)
 - [Adding a New Query](#)
- [Pushing Changes to the Dashboard](#)
 - [Step 1 — Edit and Test Locally](#)
 - [Step 2 — Copy to the Shared Drive](#)
 - [Step 3 — Copy to the Server](#)
 - [Step 4 — Restart the Dashboard](#)
- [Adding a New Chart to the Dashboard](#)
 - [Step 1 — Create the Chart Module](#)
 - [Step 2 — Register the Chart in `init.py`](#)
 - [Checklist](#)
 - [How the Frontend Renders Charts](#)
- [Adding and Removing Traders](#)
 - [Adding a Trader](#)
 - [Removing a Trader](#)
 - [Updating the Transfer Trades Excel File](#)
 - [Individual Trader Statistics](#)

- [Contacts](#)
-

Initial Setup

For those responsible for maintaining the dashboard in the future, the following steps should be followed to ensure you can push changes to the repository.

Cloning the Repository

Create the following folder on your machine:

```
C:\usr\dashboard\folder\location
```

Then clone the repository into it:

```
git clone <REPOSITORY_URL> C:\usr\dashboard\folder\location
```

If you need permission to access the repository, contact the engineering team.

Setting Up the Virtual Environment

The dashboard must run in the **virtual_env** virtual environment using **Python 3.12**. This is the same environment used by the Jupyter Playground server, so testing locally in this environment ensures compatibility before pushing to the server.

First Time Setup

1. Ensure **Python 3.12** is installed on your machine. You will also need the **Python Launcher (py)** installed.
2. Create the following directory and store the **requirements.txt** file from the repository there:

```
C:\Users\<username>\venv\
```

3. Open Command Prompt, navigate to that directory, and create the virtual environment:

```
cd C:\Users\<username>\venv\  
py -3.12 -m venv virtual_env
```

4. Activate the virtual environment:

```
virtual_env\Scripts\activate
```

5. Install dependencies:

```
pip install -r requirements.txt
```

Updating the Environment

If the `requirements.txt` file has changed (e.g. after pulling new changes from the repository), update your environment as follows:

1. Replace the existing `requirements.txt` in `C:\Users\<username>\venv\` with the updated version from the repository.
2. Activate the virtual environment:

```
virtual_env\Scripts\activate
```

3. Update dependencies:

```
pip install -r requirements.txt --upgrade
```

Keeping `requirements.txt` Up to Date

The `requirements.txt` file is stored in the root of the repository and should be updated every time you add or remove a dependency. To update it, activate your virtual environment and run:

```
pip freeze > requirements.txt
```

This command writes the exact version of every installed package to `requirements.txt`. Committing this file ensures that anyone setting up the environment in the future installs the exact same package versions.

Note: Run `pip freeze > requirements.txt` from the project root so the file is saved in the right place, then stage and commit it along with whatever other changes you are pushing:

```
git add requirements.txt
git commit -m "update requirements"
git push
```

Running the Dashboard Locally

To test changes locally before pushing to the server, activate the `virtual_env` virtual environment and run the following:

1. Open a terminal and navigate to the project root:

```
cd C:\usr\dashboard\folder\location
```

2. Activate the virtual environment:

```
virtual_env\Scripts\activate
```

3. Run the app as a module rather than a file:

```
python -m project.location.kpi_dashboard.app
```

Note: The dashboard must be run using `python -m` from the project root rather than running `app.py` directly. Running `app.py` directly will cause a `ModuleNotFoundError` because Python will not be able to resolve the package path.

The dashboard will be available at <http://localhost:5000>. Always test your changes locally before pushing to the server.

Startup Guide

Accessing the Server

The dashboard runs on a dedicated internal server. To access it, open the **Remote Desktop Connection** application on your machine and enter the server address when prompted. Login credentials can be obtained from the Trading Operations Manager.

Once logged in, you will have access to the server's file system and terminal.

Starting the Dashboard

The dashboard is installed on the server's C: drive at:

```
C:\server\dashboard_location
```

The dashboard application itself lives in the `kpi_dashboard` folder:

```
C:\server\dashboard_location\kpi_dashboard
```

To start the dashboard, navigate to this folder in the file explorer and double click `_run_kpi_dashboard.vbs`. This will silently start the Waitress server in the background with no terminal window. This is done intentionally as multiple people use the server and we do not want the dashboard to be accidentally shut down by closing a terminal window.

The `.vbs` file contains the following:

```
CreateObject("Wscript.Shell").Run "cmd /c _run_kpi_dashboard.bat", 0, False
```

Note: If there is already an instance of the dashboard running, **kill it** before starting a new one. See the [Killing the Dashboard](#) section below.

Note: If the dashboard is not working or needs to be debugged, simply run `_run_kpi_dashboard.bat` directly instead. This will open a terminal window and print any errors to the console. If the programme is running correctly you should see the following output:

```
[1/4] Changing to project root...
      CWD = C:\folder\location
[2/4] Activating venv...
[3/4] Ensuring waitress is installed...
[4/4] Starting waitress on 0.0.0.0:5000 ...
```

Once the server is running, the dashboard is accessible to anyone on the internal network at the configured server URL.

Note: The server holds all data in memory. If the server is restarted for any reason, users will need to reload their data by clicking **Load Data** again, as data does not persist between server restarts.

IMAGE REDACTED: Screenshot of Dashboard Loaded in Browser

Killing the Dashboard

The dashboard runs as a background `python.exe` process in Task Manager. To kill it:

1. Open **Task Manager** (`Ctrl + Shift + Esc`)
2. Go to the **Details** tab
3. Look for active `python.exe` processes
4. To identify the correct one, go to the dashboard in your browser and either:
 - Click **Load Data**, or
 - Toggle between the different group tabs
5. Watch the **CPU** column in Task Manager - the correct `python.exe` process will light up when the dashboard is active
6. Right click the correct `python.exe` and select **End Task**

Note: Be careful not to kill the wrong `python.exe` if there are other Python processes running on the server.

IMAGE REDACTED: Screenshot of the Server's Task Manager

Project Structure & Data Flow

Directory Structure

```

tops_dashboards/
├── kpi_dashboard/
│   └── app.py                # Flask application, API endpoints, stats
calculations
├── data_loader.py           # KPIDataManager class, all data loading and query
logic
├── global_functions.py      # Reusable query helpers and market data functions
├── charts/
│   ├── __init__.py          # Chart registry, section definitions, routing
│   ├── colors.py            # Shared ElectroRoute colour palette
│   ├── traders.py           # Lists containing all active traders
│   ├── auction_prices.py    # Example chart module
│   └── ...                  # One file per chart
├── static/
│   ├── electroroute_logo.png
│   └── electroroute_logo_small.png
├── templates/
│   └── index.html           # Single-page frontend, all rendering is dynamic
├── README.md
└── _run_kpi_dashboard.bat   # Windows launcher script

```

IMAGE REDACTED: File structure as seen in VSCode

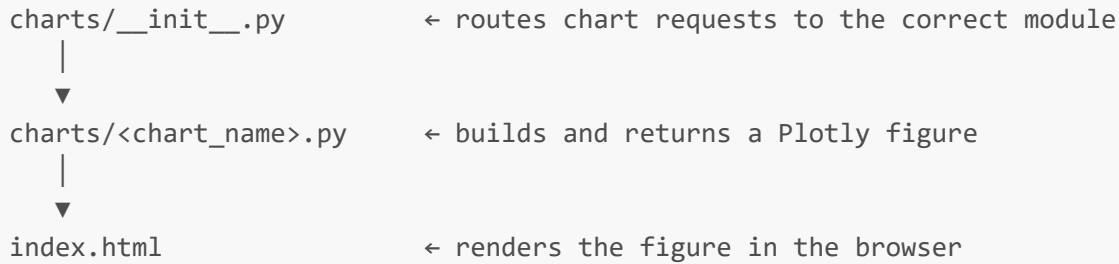
How Data Flows Through the Dashboard

The dashboard follows a linear pipeline from database to browser. Each stage is described below.

```

Databases
  |
  ▼
global_functions.py    ← reusable query helpers
  |
  ▼
data_loader.py         ← KPIDataManager loads and processes all data
  |
  ▼
app.py (dashboard_state) ← processed results held in memory
  |
  ▼

```



Stage 1 — Database Connections (`global_functions.py`)

`global_functions.py` provides all reusable functions for querying the databases and retrieving market data. It does not contain any business logic specific to a group or chart — it is a shared library used by `data_loader.py`.

Available Databases

Database	Query Function	Contents
Market Data DB	<code>run_market_data_query</code>	Market prices, forecasts, SNSP
Legacy Trades DB	<code>run_legacy_trades_query</code>	Legacy trading positions and deals
Current Trades DB	<code>run_current_trades_query</code>	Current trading positions and deals
Settlement DB	<code>run_settlement_query</code>	Settlement reports, statements, imbalance prices
Unit Mapping DB	<code>run_unit_mapping_query</code>	Unit-to-group mappings and contract dates

All query functions accept a raw SQL string and an `instance_role` argument (always `"rep-1"` for the read replica).

Key Functions Available

Function	Description
<code>all_isem_prices(start, end)</code>	Returns DA, IDA1, IDA2, IDA3 and imbalance prices at 30-min resolution
<code>irl_da_price_vol(start, end)</code>	DA prices only, handles hourly/30-min MTU transition
<code>irl_ida_price_vol(start, end, IDA_run)</code>	IDA prices for a specific run (1, 2 or 3)
<code>irl_imb_settle_price(start, end)</code>	Imbalance settlement prices and NIV
<code>irl_SNSP(start, end)</code>	System Non-Synchronous Penetration percentage
<code>unit_forecast_DA(group, source, start, end)</code>	DA generation forecasts for a group
<code>unit_forecast_IDA1/2/3(group, source, start, end)</code>	IDA generation forecasts for a group

Function	Description
<code>position_dataframe(group, start, end)</code>	Combined trading positions from multiple sources
<code>irl_settlement_reports_v2(start, end, group)</code>	Physical settlement quantities (QM, QAB, etc.)
<code>irl_settlement_statements_v2(start, end, group)</code>	Financial settlement charges in EUR
<code>irl_settlement_reports_unstack(start, end, group)</code>	Settlement reports in long format, one row per unit
<code>irl_portfolio_info(start, end, group)</code>	Unit details and contract dates for a group
<code>market_data_pull(...)</code>	Generic low-level function to pull any data from the market data database
<code>fx_call(start, end)</code>	GBP to EUR exchange rates at 30-min resolution

All time series data is returned as a pandas DataFrame with a datetime index in Irish Standard Time (IST) unless otherwise specified.

Stage 2 — Data Loading (`data_loader.py`)

`data_loader.py` contains the `KPIDataManager` class which orchestrates all data loading. A single instance of this class is created at startup and reused for the lifetime of the server process.

```
_data_manager = None

def get_data_manager():
    global _data_manager
    if _data_manager is None:
        _data_manager = KPIDataManager()
    return _data_manager
```

When the user clicks **Load Data**, `app.py` calls `load_all_data(start_date, end_date, agg_freq)` which:

1. Sets the date range and aggregation frequency on the data manager
2. Calls `load_common_data()` — fetches prices and SNSP once, shared across all groups
3. Calls `load_group_data(grp)` for each configured group
4. Calls `load_trader_data()` — fetches trader-specific IDA and IDC data

What `load_group_data` Does

For each group, `load_group_data` queries all required data, joins it into a single DataFrame `df`, and computes all derived columns. The key steps are:

1. Queries installed capacity from unit mapping
2. Queries DA, IDA1, IDA2, IDA3 forecasts from configured forecast providers

3. Queries trading positions (combines multiple data sources)
4. Queries settlement reports (`QM`, `QAB`, `QABCURL`, etc.) and settlement charges (`CIMB`, `CCURL`, etc.)
5. Joins everything into a single half-hourly DataFrame indexed by IST datetime
6. Computes derived columns including:
 - `ExA_Income` — total exchange auction income (`DA` + `IDA1` + `IDA2` + `IDA3` + `IDC`)
 - `IMB_Income` — imbalance income including BM charges
 - `Net_Income` — total income
 - `Client_Payment` — payment owed to the client based on DA price
 - `Cost` — balancing cost (`Client_Payment` - `Net_Income`)
 - `100_Forecast_Cost`, `90_Forecast_Cost`, `110_Forecast_Cost` — scenario costs
 - `IDA1_Gain`, `IDA2_Gain`, `IDA3_Gain`, `IDC_Gain` — net gain from each auction
 - `Net_Gain`, `Cumulative_IDA_Gain` — total and running IDA gain

Stage 3 — The `results` Object

After loading, all data is stored in `dashboard_state["results"]` in `app.py`. This is a dictionary keyed by group name:

```
results = {
    "DAM": { ... },    # Group data dict
    "RFIT": { ... },
    "SOL": { ... },
    "TRADER": { ... }, # Trader-specific data
}
```

Group Data Dict Structure

Each group entry (e.g. `results["DAM"]`) contains:

Key	Type	Description
<code>df</code>	<code>DataFrame</code>	Main half-hourly time series, all columns listed below
<code>start</code>	<code>date</code>	Actual start date of data
<code>end</code>	<code>date</code>	Actual end date (clipped to latest settlement)
<code>agg_freq</code>	<code>str</code>	Aggregation frequency (" <code>D</code> " or " <code>MS</code> ")
<code>agg_label</code>	<code>str</code>	Human-readable label (" <code>Daily</code> " or " <code>Monthly</code> ")
<code>Capacity</code>	<code>Series</code>	Installed capacity at 30-min resolution
<code>Total_Prod</code>	<code>float</code>	Total production in MWh
<code>Total_Cost</code>	<code>float</code>	Total balancing cost in EUR
<code>Cost_per_MWh</code>	<code>float</code>	Balancing cost per MWh
<code>DA_F_Total_Cost</code>	<code>float</code>	Cost if 100% DA forecast had been traded

Key	Type	Description
DA_F_Cost_per_MWh	float	Above, per MWh
DA_F90_Total_Cost	float	Cost at 90% DA forecast scenario
DA_F110_Total_Cost	float	Cost at 110% DA forecast scenario
DA_pos_mwh	float	DA position balancing cost per MWh
Agg_DA_pos_Cost	Series	DA position cost aggregated to <code>agg_freq</code>
q_Plot	DataFrame	Generation quantities for plotting (MW scale)
q_IMB	DataFrame	Imbalance quantity
q_Volume	DataFrame	Traded and net volume
Trade_Ratio	DataFrame	Ratio of traded to available volume

Main `df` Columns

The `df` inside each group result contains the following columns at 30-min resolution:

Column	Description
QM	Metered quantity
QMLF	Metered quantity adjusted for loss factors
QAB	Total dispatch down
QABCURL	Curtailement component of dispatch down
QABNF	Non-firm dispatch down
QABUNDEL	Under-delivery dispatch down
QEX	Traded (ex-ante) quantity
QFPN	Final physical notification
QD	Demand
LF	Loss factor
PDA	Day-Ahead price
PIDA1	IDA1 clearing price
PIDA2	IDA2 clearing price
PIDA3	IDA3 clearing price
PIMB	Imbalance settlement price
NIV	Net imbalance volume
SNSP	System Non-Synchronous Penetration

Column	Description
DA_MW	DA position in MW
IDA1_MW	IDA1 position in MW
IDA2_MW	IDA2 position in MW
IDA3_MW	IDA3 position in MW
IDC_MW	IDC position in MW
DA_EUR	DA auction income
IDA1_EUR	IDA1 auction income
IDA2_EUR	IDA2 auction income
IDA3_EUR	IDA3 auction income
IDC_EUR	IDC income
DA_Forecast_Provider_A	DA forecast from provider A (loss factor adjusted)
IDA1/2/3_Forecast_Provider_A	IDA forecasts from provider A
DA_Forecast_Provider_B	DA forecast from provider B
IDA1/2/3_Forecast_Provider_B	IDA forecasts from provider B
ExA_Income	Total exchange auction income
IMB_Income	Imbalance income
Net_Income	Total net income
Client_Payment	Payment owed to client
Cost	Balancing cost
IDA1/2/3_Gain	Net gain from each IDA auction vs imbalance
IDC_Gain	Net gain from IDC vs imbalance
Net_Gain	Total IDA + IDC gain
Cumulative_IDA_Gain	Running cumulative gain
100/90/110_Forecast_Cost	Scenario balancing costs
DA_pos_cost	Cost of the DA position only
trading_day	Normalised trading day date
Price_Spread	PIMB - PDA

Trader Data Dict Structure

results["TRADER"] contains:

Key	Type	Description
dfIDA	DataFrame	Raw IDA positions, prices and rota assignments
dfIDC	DataFrame	IDC trades with rota assignments and profit
dfIDA_pnl	DataFrame	Calculated PnL per auction per half-hour
imb_df	DataFrame	Imbalance prices for the period
start	date	Start date
end	date	End date

Stage 4 — Chart Rendering (`charts/`)

When the frontend requests a chart via `/api/chart/<chart_id>`, `app.py` calls `create_chart()` from `charts/__init__.py`, passing the full `results` dictionary along with the selected group, aggregation frequency and date range.

The chart module receives `results` and is responsible for extracting what it needs. Most group charts follow this pattern:

```
def create_chart(results, group, agg_freq, start_str, end_str):
    data = results.get(group)

    if data is None:
        fig = go.Figure()
        fig.add_annotation(text="No data available", showarrow=False)
        return fig, ""

    df = data["df"]
    # use df columns, data["Capacity"], data["agg_freq"], etc.
```

Charts that are not group-specific (e.g. `CHART_GROUPS = ["MARKET"]`) should loop through available groups to find data:

```
for g in ["DAM", "RFIT", "SOL"]:
    if g in results and results[g] is not None:
        data = results[g]
        break
```

Trader charts should access `results["TRADER"]` directly:

```
trader_data = results.get("TRADER")
dfIDA_pnl = trader_data["dfIDA_pnl"]
dfIDC = trader_data["dfIDC"]
```

Adding a New Query

If your chart requires data that is not already in `df`, there are two options depending on the nature of the data.

Option A — Add to `load_group_data` in `data_loader.py`

Use this if the data is needed by multiple charts or is fundamental to the group's analysis. Add the query inside `load_group_data`, join the result into `df`, and store any aggregated values in the group result dict.

```
# Inside load_group_data, after existing queries
my_new_data = gf.some_query_function(query_start, grp_end)
df = df.join(my_new_data, how="left")

# Store a summary value if needed
self.results[grp]["My_New_Total"] = df["my_new_column"].sum()
```

The new column will then be available in `data["df"]` in every chart module.

Option B — Query Directly in the Chart Module

Use this if the data is only needed by one specific chart and does not belong in the main `df`. Import the relevant query function from `global_functions` and call it inside `create_chart`.

```
import project.location.kpi_dashboard.global_functions as gf

def create_chart(results, group, agg_freq, start_str, end_str):
    data = results.get(group)
    start = data["start"]
    end = data["end"]

    my_df = gf.some_query_function(start, end)
    # use my_df to build the chart
```

Note: Querying directly inside a chart module means the query runs every time the chart is requested. For expensive queries, Option A is preferred as the data is loaded once and cached in memory.

Adding a New Query Function to `global_functions.py`

If neither the existing query functions nor the data already in `df` cover what you need, you can add a new function to `global_functions.py`. This is straightforward — the main thing you need to identify is **which database the data is coming from**, then use the corresponding query runner.

The following query runners are available at the top of `global_functions.py`:

```

from library.name.<path>.queries import run_query as run_market_data_query
from library.name.<path>.queries import run_query as run_settlement_query
from library.name.<path>.queries import run_query as run_legacy_trades_query
from library.name.<path>.queries import run_query as run_current_trades_query
from library.name.<path>.queries import run_query as run_unit_mapping_query

```

Runner	Database	Contains
<code>run_market_data_query</code>	Market Data	Market prices, forecasts, SNSP
<code>run_settlement_query</code>	Settlement	Settlement reports, statements, imbalance prices
<code>run_legacy_trades_query</code>	Legacy Trades	Legacy trading positions and deals
<code>run_current_trades_query</code>	Current Trades	Current trading positions and deals
<code>run_unit_mapping_query</code>	Unit Mapping	Unit-to-group mappings and contract dates

Each runner accepts a raw SQL string and an `instance_role` argument (always `"rep-1"` for the read replica) and returns a pandas DataFrame.

For queries against the market data database that follow the standard dimension model, you can also use the `market_data_pull` helper which handles the joins for you. See existing functions such as `irl_demand_actual` in `global_functions.py` for an example of this pattern.

When writing a new function, follow these conventions:

- Use the appropriate runner for the database you need
- Return a DataFrame with a datetime index in IST
- Handle the empty DataFrame case
- Name the function descriptively

A basic SQL query function looks like this:

```

def my_new_query(start, end):
    query = """
    SELECT datetime, value
    FROM some_table
    WHERE datetime >= "{0}"
    AND datetime < "{1}"
    """.format(start, end)

    df = run_settlement_query(query, instance_role="rep-1")

    if df.empty:
        return pd.DataFrame()

    df.columns = ["time", "my_value"]
    df = df.set_index("time")

```

```
df.index = pd.to_datetime(df.index)
return df
```

For data that comes from the market data database via the `market_data_pull` helper, the pattern looks like this:

```
def my_new_market_data_query(start, end, local_time=True):
    df = market_data_pull(
        start,
        end,
        segment="Power",
        category="Some|Category|Here",
        country="IE",
        source="SomeSource",
        time_interval="30 Minute",
        value_column="my_value",
        local_time=local_time,
    )

    if df.empty:
        return pd.DataFrame()

    return df
```

Once your function is written and tested, it can be called from either `data_loader.py` or directly inside a chart module as described in Options A and B above.

SSH Setup for Repository Access

If you are getting `Permission denied (publickey)` when trying to push to the repository, follow these steps.

1. Generate an SSH key (if you do not have one)

```
ssh-keygen -t ed25519 -C "your.email@company.com"
```

2. Add your public key to the repository host

- Copy the contents of `C:/Users/<username>/.ssh/<keyname>.pub`
- Go to your repository hosting platform > Personal Settings > SSH Keys > Add Key

3. Create the SSH config file

In Git Bash run:

```
touch ~/.ssh/config  
notepad ~/.ssh/config
```

Paste in the following and save:

```
Host <repository_host>  
  HostName <repository_host>  
  User git  
  IdentityFile C:/Users/<username>/.ssh/<keyname>  
  IdentitiesOnly yes
```

4. Tell Git to use Windows OpenSSH

```
git config --global core.sshCommand "C:/Windows/System32/OpenSSH/ssh.exe"
```

5. Test the connection

```
ssh -T git@<repository_host>
```

You should see a message confirming authentication.

6. Push your changes

```
git push
```

Note: Replace **<username>** with your Windows username and **<keyname>** with your SSH key name. You may need to contact IT if you need the Windows OpenSSH agent service enabled as it requires admin access.

Pushing Changes to the Dashboard

Due to the dashboard being hosted on a server, changes cannot be deployed in the same way as a typical local project. Instead, files are manually transferred from your local machine to the server via a shared network drive. To prevent breaking the live dashboard, only authorised users should be permitted to push changes.

The full process involves three locations:

- Your **local machine** (where changes are made and tested)
- The **shared network drive** (used as an intermediary, only accessible on the VPN)
- The **dashboard server** (where the live dashboard runs)

Note: If a broken chart is pushed to the server, it will not crash the entire dashboard. Each chart loads independently, so a broken chart will show an error box in its place while all other charts continue to render normally. That said, always test locally before pushing.

Step 1 — Edit and Test Locally

Open the project on your local machine in your chosen IDE (VS Code, PyCharm, Cursor, etc.):

```
C:\usr\dashboard\folder\location
```

Before making any changes, ensure your local copy is up to date by running the following in your terminal:

```
git pull
```

Make your changes, test them locally, and once you are happy, stage, commit, and push them to the repository:

```
git add .  
git commit -m "your commit message"  
git push
```

Step 2 — Copy to the Shared Drive

Once your changes are pushed, copy the updated **tops_dashboards** folder from your local C: drive to the shared network drive, replacing the existing folder at the designated shared location.

The contents of this folder should always mirror those in your local **C:\usr\dashboard\folder\location** folder, minus any untested changes.

IMAGE REDACTED: Copying from local C: drive to shared drive

Step 3 — Copy to the Server

Log into the dashboard server via **Remote Desktop Connection**. On the server desktop, open two file explorer windows side by side:

- One pointing to the **shared network drive** at the designated shared location
- One pointing to the **server's C: drive** at **C:\er_apps**

Copy the **tops_dashboards** folder from the shared drive and paste it into **C:\er_apps** on the server, replacing the existing folder.

IMAGE REDACTED: Copying from shared drive to server C: drive

Step 4 — Restart the Dashboard

Navigate to the following folder on the server:

```
C:\server\dashboard_location\kpi_dashboard
```

Kill the existing dashboard process if one is running (see [Killing the Dashboard](#)), then double click `_run_kpi_dashboard.vbs` to start the server.

Once the server is running, open the dashboard in a browser and click **Load Data** to confirm everything is working as expected.

Adding a New Chart to the Dashboard

Each chart in the dashboard is a self-contained Python module stored in the `charts/` folder. To add a new chart, the following steps must be completed:

1. Create the chart module
2. Register it in `charts/__init__.py`

Step 1 — Create the Chart Module

Create a new `.py` file in the `charts/` folder. The filename should be descriptive and use underscores, for example:

```
charts/my_new_chart.py
```

Every chart module must define the following metadata constants at the bottom of the file:

```
CHART_ID = "my_new_chart"           # Unique identifier for the chart
CHART_TITLE = "My New Chart"        # Display title shown in the dashboard
CHART_SECTION = "market_prices"     # Section the chart belongs to (see SECTIONS
in __init__.py)
CHART_GROUPS = ["DAM", "RFIT"]      # Groups that can view this chart
CHART_DESCRIPTION = "A brief description of what this chart shows"
CHART_AGG_FREQUENCIES = None        # Set to a list of frequencies if applicable,
otherwise None
```

Note: `CHART_ID` must be unique across all chart modules. If two charts share the same ID, one will overwrite the other in the registry.

Available Sections

The following sections are currently defined in `charts/__init__.py`. Assign your chart to the most appropriate one:

Section ID	Title	Tab
<code>ida_trading</code>	IDA Trading Performance	Group
<code>balancing_costs</code>	Balancing Cost Scenarios	Group
<code>position_analysis</code>	Position Analysis	Group
<code>imbalance_analysis</code>	Imbalance Analysis	Group
<code>market_prices</code>	Market Prices	Market
<code>forecasts</code>	Forecasts & Accuracy	Market
<code>trader_overview</code>	Trader Overview	Trader
<code>trader_decisions</code>	Trading Decisions Analysis	Trader
<code>trader_performance</code>	Performance Over Time	Trader
<code>summary</code>	Summary Statistics	Other
<code>cross_group</code>	Cross-Group Comparison	Other

If none of the existing sections are appropriate, add a new one to the `SECTIONS` dictionary in `charts/__init__.py`:

```
"my_new_section": {"title": "My New Section", "order": 5},
```

The `order` value determines where the section appears on the page relative to other sections within the same tab.

Available Groups

Charts can be assigned to one or more of the following groups:

Group	Description
<code>DAM</code>	Day Ahead Market group
<code>RFIT</code>	RFIT group
<code>SOL</code>	SOL group
<code>MARKET</code>	Market-wide data, not group specific

The `create_chart` Function

Every chart module must implement a `create_chart` function with the following signature:

```
def create_chart(results, group, agg_freq, start_str, end_str):
```

Parameter	Description
<code>results</code>	Dictionary of data keyed by group (DAM, RFIT, SOL)
<code>group</code>	The currently selected group
<code>agg_freq</code>	The selected aggregation frequency
<code>start_str</code>	Start date as a string
<code>end_str</code>	End date as a string

The function must return a tuple of:

```
return fig, summary_html
```

- `fig` — a Plotly `Figure` object
- `summary_html` — an HTML string containing a summary table, or an empty string `""` if no summary is needed

Always handle the case where no data is available by returning an empty figure with an annotation:

```
if data is None:
    fig = go.Figure()
    fig.add_annotation(text="No data available", showarrow=False)
    return fig, ""
```

Colors

Use the shared color palette imported from `charts/colors.py` to keep charts visually consistent:

```
from .colors import ER_BLUE
from .colors import ER_DARK_GREEN
from .colors import ER_LIGHT_GREEN
from .colors import ER_ORANGE
from .colors import ER_PURPLE
```

Step 2 — Register the Chart in `__init__.py`

Open `charts/__init__.py` and make the following two additions:

1. Import the module at the top of the file:

```
from . import my_new_chart
```

2. Add it to the **CHART_MODULES** list:

```
CHART_MODULES = [
    ...
    my_new_chart,
]
```

The order of modules in **CHART_MODULES** determines the order charts appear within their section on the dashboard.

Checklist

Before pushing your new chart, confirm the following:

- ☐ Chart module created in **charts/**
- ☐ **CHART_ID** is unique
- ☐ **CHART_SECTION** exists in **SECTIONS** or a new section has been added
- ☐ **CHART_GROUPS** is set correctly
- ☐ **create_chart** returns a **(fig, html)** tuple
- ☐ No data case is handled gracefully
- ☐ Module imported and added to **CHART_MODULES** in **__init__.py**
- ☐ Chart tested locally before pushing

How the Frontend Renders Charts

The **index.html** template does not need to be modified when adding a new chart. The frontend is fully dynamic and discovers charts automatically at runtime by calling the following backend API endpoints:

Endpoint	Description
/api/sections/<group>	Returns all sections and their charts for a given group
/api/chart/<chart_id>	Returns the Plotly figure and summary HTML for a given chart
/api/stats/<group>	Returns the summary statistics widgets for a given group

When a user switches tab or loads data, the frontend calls **/api/sections/<group>** which internally calls **get_sections_for_group()** from **charts/__init__.py**. This scans the **CHART_MODULES** list, reads each module's metadata constants (**CHART_ID**, **CHART_TITLE**, **CHART_SECTION**, **CHART_GROUPS**, **CHART_DESCRIPTION**), and returns a structured list of sections and charts for that group.

The frontend then dynamically builds the page layout — rendering a card for each chart it receives — and fires off a separate request to **/api/chart/<chart_id>** for each one. Each chart loads independently, so a slow or broken chart will not block the others from rendering.

This means that as long as your new chart module is:

- Imported in `charts/__init__.py`
- Added to `CHART_MODULES`
- Assigned to a valid `CHART_SECTION` and `CHART_GROUPS`

It will appear on the dashboard automatically with no further changes required.

Adding and Removing Traders

As new traders join and others leave, the list of active traders will need to be kept up to date. Thankfully the code is laid out in a way that makes this fairly straightforward.

All trader configuration is stored in a single file:

```
kpi_dashboard/traders.py
```

This is the **only file that needs to be modified** when adding or removing a trader. It contains three dictionaries that need to be kept in sync with each other:

Variable	Description
<code>ACTIVE_TRADERS</code>	List of full trader names used throughout the dashboard
<code>TRADER_COLOURS</code>	Maps each trader's full name to a hex colour for charts
<code>TRADERS</code>	Maps each trader's first name (as it appears in the database) to their full name

Adding a Trader

1. Add their full name to `ACTIVE_TRADERS`:

```
ACTIVE_TRADERS = [  
    ...  
    "New Trader",  
]
```

2. Assign them a colour in `TRADER_COLOURS`:

```
TRADER_COLOURS = {  
    ...  
    "New Trader": "#e41a1c",  
}
```

3. Add their first name mapping to `TRADERS`. The key must match exactly how their name appears in the database rota:

```
TRADERS = {
    ...
    "New": "New Trader",
}
```

Removing a Trader

Remove their entry from all three — `ACTIVE_TRADERS`, `TRADER_COLOURS`, and `TRADERS`.

Note: Make sure to update all three dictionaries together. If a trader appears in one but not the others it will cause errors in the dashboard.

Updating the Transfer Trades Excel File

In addition to `traders.py`, there are two other files that will need to be updated on a yearly basis. At the top of `data_loader.py`, the following paths are defined:

```
TRANSFER_EXCEL_PATH_CURRENT_FY = (
    "<SHARED_DRIVE_PATH>\\Shift_team_transfer_trades_currentFY.xlsx"
)
TRANSFER_EXCEL_PATH_NEXT_FY = (
    "<SHARED_DRIVE_PATH>\\Shift_team_transfer_trades_nextFY.xlsx"
)
NEXT_FY_START = dt.date(<year>, 4, 1)
```

These point to Excel files on the shared drive that contain manually recorded transfer trades between traders. Two files are always maintained - one for the current financial year and one for the previous financial year. This is intentional, as analysis may span across two financial years and both files are needed to ensure the full picture is captured.

The filename includes the financial year. At the start of each new financial year, a new Excel file will be created for that year. When this happens:

1. The older of the two paths should have its filename updated to the previous financial year's file
2. The newer path should be updated to point to the new financial year's file
3. The `FY_START` variable should be updated to reflect the start date of the new financial year (1st April)

Note: If either path is wrong or the file cannot be found, the dashboard will log a warning but will not crash. Transfer trades will simply not be applied for that period. You will see the following message in the terminal:

```
Warning: Could not load transfer trades from <path>: ...
```

Individual Trader Statistics

The individual trader statistics shown on the dashboard are derived by cross-referencing the trading rota with the trades that were executed during each trader's shift. The rota is pulled from the rota database table, which records which trader was on the day and night shift for each half-hour period. This is then matched against the trades executed during that time to attribute performance to the correct trader.

For IDC trades specifically, transfer trades recorded in the Excel files above are used to correct any trades that were executed by one trader on behalf of another, ensuring the statistics accurately reflect each trader's individual performance.

Once you have made your changes to either file, test locally and then follow the [Pushing Changes to the Dashboard](#) steps to deploy the update to the server.

Contacts

Role	Responsible For
Trading Operations Manager	Server login credentials, dashboard content and chart decisions
Engineering Lead	Repository access, server infrastructure, database permissions
Finn O'Connor	Built the dashboard, wrote this documentation